Cache associativity

Sparsh Mittal

An example: bookshelf

- Assume I have a bookshelf with N spaces
- I can organize it differently
- **Organization 1: Keep a book anywhere**

- Plus: ease of arranging, full utilization of capacity
- Minus: more time in retrieving (searching)

Organization 2: Make two partitions

Organization 2: Make two partitions

Partition 1: research books

Partition 2: teaching books

Plus: time required for searching reduced

Minus: >N/2 research books cannot fit, even if teaching books are <N/2. Thus, full capacity is not utilized

Organization 3: Make N partitions

Organization 3: Make N partitions

A book can go to only one space, belonging to its topic.

Plus: time required for searching constant

Minus: >1 book on any topic cannot fit

Spectrum of Associativity

Direct-mapped (1 way, k-sets)

Set-associative (m way, k/m sets)

Fully-associative (1 set, k-way)

Spectrum of Associativity

For a cache with 8 entries

One-way set associative (direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

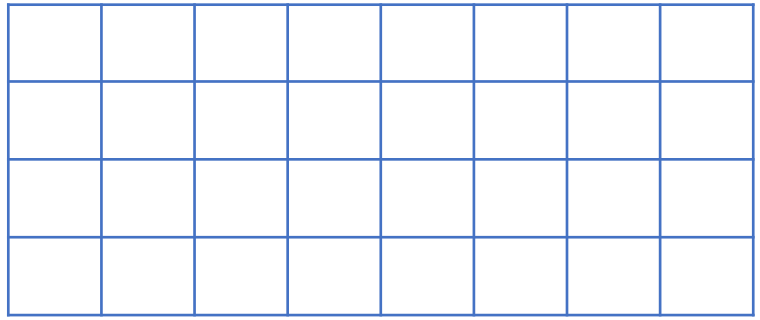
# of ways = associativity

Understanding sets/ways/blocks

Consider any table:

8 columns

4 rows



Number of cells = #Rows * #Columns = 32

Similarly, in cache, we call:
#Rows as #Sets
#Columns as #Ways (or associativity)
#Cells is same as #Blocks

Cache organization

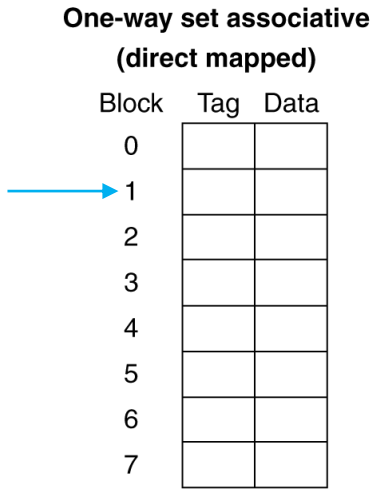
- Cache size
- = (# of blocks) * (block size)
- = (# of sets) * (# of ways) * (block size)

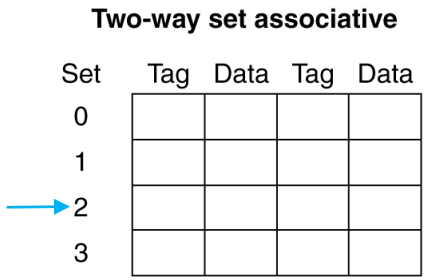
- Example: block size = 64B, # of ways =16, cache size = 4MB
- => # of sets = 4096

- A Block is also called a line/element/entry

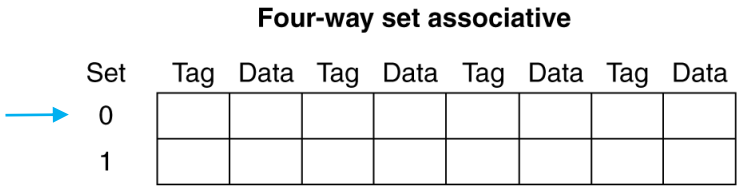
How many ways are searched in each cache design

- Assume we access any set in each cache.
- How many ways are searched in that cache?
- Answer: all the ways.
- ➔ Higher associativity cache will have higher search overhead.

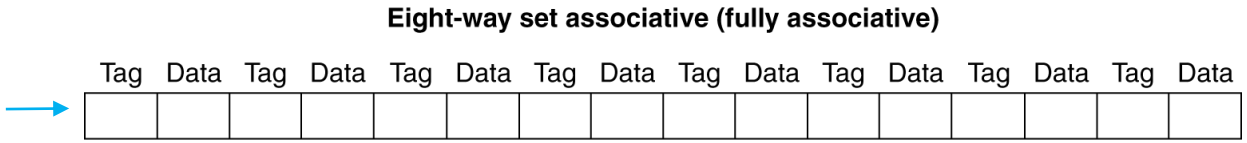




Both ways of this set are searched



All 4 ways of this set are searched



All 8 ways of this set are searched

# of ways = associativity

Associative Caches

Fully associative

- Allow a given block to go in any cache entry
- Requires all entries to be searched at once
- 1 Comparator per entry (expensive)

n-way set associative

- Each set contains n entries
- Search all entries in a given set at once
- n comparators (less expensive)

Understanding Tag

Consider a table for searching numbers

We use decimal number system

This table has 10 rows, one each for 0 to 9

0				
1				
2				
3				
4				
5				
6				
7				
8				
9				

Now store numbers in this table

We have: 409, 318, 444, 981, 984, 004, 290, 716

Store each number in the row based on its least-significant digit

0	290			
1	981			
2				
3				
4	984	444	004	
5				
6	716			
7				
8	318			
9	409			

Can we save some space?

Key idea: If a number is stored in row number, say, 4, then, its one's place will surely be 4. Hence, no need to store that explicitly

0	29			
1	98			
2				
3				
4	98	44	00	
5				
6	71			
7				
8	31			
9	40			

Observe: It was perfectly OK to store the full number.
But, to save space, we stored part of the number, instead of whole number. This did not create any error or info-loss.
This "part of number" is called a **tag**. Full number is **address**.
Tag 98 in '1' and '4' row are same: no problem!

Tag and valid bits

How do we know which particular block is stored in a cache location?

- Store block address as well as the data
- We only need the high-order bits called tag

What if there is no data in a location?

- Valid bit: 1 = present, 0 = not present
- Initially 0

Offset is also called block offset

Address division in cache

Let memory address be of N bits. First find S and F

S = log2 (#Sets)

F = log2(BlockSize)

T = N-S-F

Example: A cache has 4096 sets and 64B block size. Memory addresses are 32b. Find S, F, N and T.

Answer: N=32 bits. F = log2 (64) = 6 bits. S = log2 (4096) = 12 bits

T = 32-6-12 = 14 bits

Special cases

Case 1: Block size is 1B ➔ F=0

Case 2: Only one set (fully-associative cache) ➔ S=0

Case 3: Block size is 1B and only one set ➔ S=0 and F=0 ➔ Address is same as the tag

Case 4: If cache size is same as memory size, then T=0. Note that this does not happen in practice.

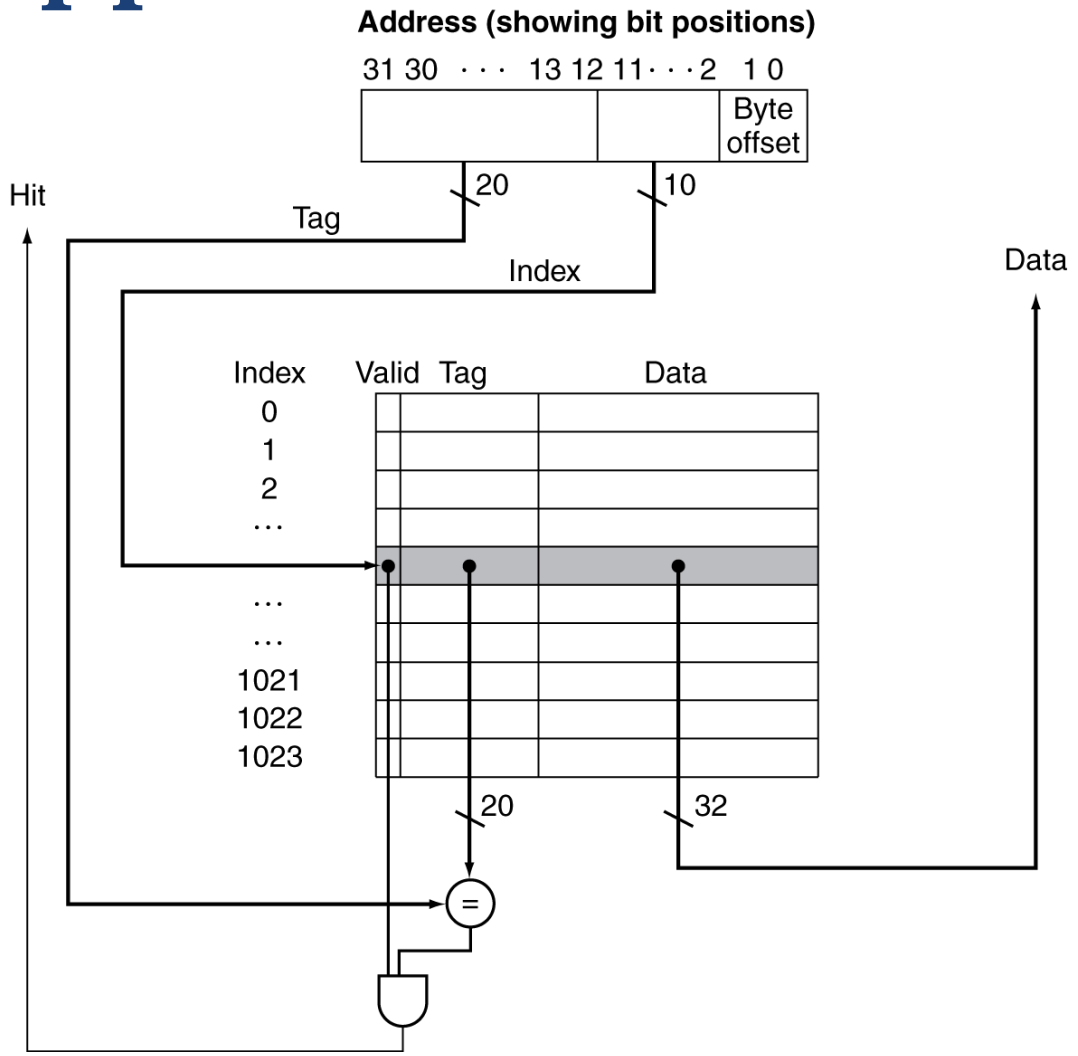
Uniqueness of tags

Within a single set, the tags stored in different ways are unique.

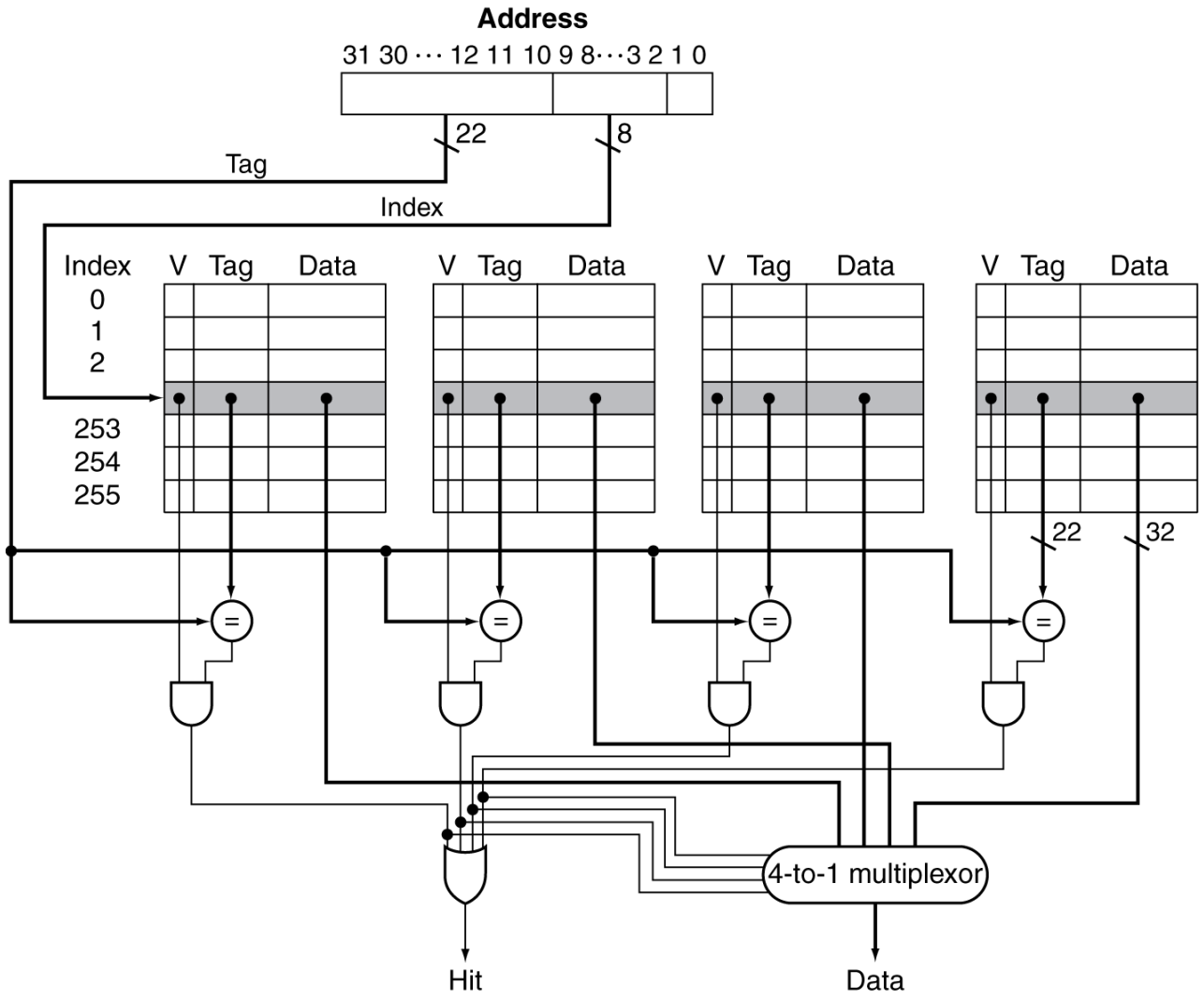
The tags stored in different sets could be same or different.

How a data-block is searched in different caches

Direct-mapped Cache



Set Associative Cache



Solved example

Associativity Example

- A cache sees following addresses: 0, 8, 0, 6, 8
- It has only 4 blocks, each 1B size
- There are three design choices:
  - Direct mapped (1-way set associative)
  - 2-way set associative
  - Fully associative (i.e., 4-way set associative)
- Show cache operation in each case

Associativity Example

First compute the set in which an address will be stored.

This is not required for fully-associative cache, since it has only 1 set.

Block-size is 1B, so there is no bit in block-offset.

With 4-blocks, direct-mapped cache has 4 sets, so 2 lower bits will be used for finding set-index.

With 4-blocks, 2-way associative cache has 2 sets, so 1 lowest bit will be used for finding set-index.

		Set (index)		
Address	Address (Binary)	direct-mapped	2-way	fully-assoc
0	0000	00	0	
6	0110	10	0	
8	1000	00	0	

Associativity Example

Direct mapped

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0	miss	**Mem[0]**			
8	0	miss	**Mem[8]**			
0	0	miss	**Mem[0]**			
6	2	miss	Mem[0]		**Mem[6]**	
8	0	miss	**Mem[8]**		Mem[6]	

Associativity Example

- 2-way set associative

Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
0	0	miss	**Mem[0]**			
8	0	miss	Mem[0]	**Mem[8]**		
0	0	hit	**Mem[0]**	Mem[8]		
6	0	miss	Mem[0]	**Mem[6]**		
8	0	miss	**Mem[8]**	Mem[6]		

- Fully associative

Block address		Hit/miss	Cache content after access			
0		miss	**Mem[0]**			
8		miss	Mem[0]	**Mem[8]**		
0		hit	**Mem[0]**	Mem[8]		
6		miss	Mem[0]	Mem[8]	**Mem[6]**	
8		hit	Mem[0]	**Mem[8]**	Mem[6]	